

Professor — Tutor de código proativo e local (Design)

- **Data:** 2026-06-26
- **Status:** Design aprovado em conversa; aguardando revisão deste documento antes do plano de implementação.
- **Autor:** juaredbr + Claude

1. Visão

Um **tutor de programação pessoal**, proativo, dentro do VSCode, rodando na **GPU local (RTX 4080 via Ollama / Qwen3 14B)**. Ele observa o código enquanto você escreve e, em checkpoints naturais, oferece **dicas didáticas** num **painel lateral**, fundamentadas em soluções reais do **StackOverflow offline** (arquivo `.zim` no servidor). É o "delta" que falta entre o autocomplete atual e um professor que ensina na prática.

2. Objetivos e não-objetivos

Objetivo nº1 (decidido): *ensinar a programar melhor* — pedagogia acima de produtividade. A ferramenta **guia**; não faz o trabalho por você.

Objetivos: - Dicas **proativas** e didáticas, num **painel lateral** nativo do VSCode. - Fundamentação em **StackOverflow offline** (com citação + link). - 100% **local** (4080 + servidor da LAN); sem nuvem. - **Não atrapalhar**: silencioso quando não há o que ensinar; degradação graciosa.

Não-objetivos (YAGNI): - **Não** é um autocomplete melhor (isso é problema separado; o Continue já cobre). - **Não** gera código por você por padrão. - **Não** é um chat (o Continue já oferece, se quiser manter em paralelo). - O **MVP não tem RAG vetorial** — usa busca direta no `.zim`. Vetorial vem depois.

3. Arquitetura



Princípio: a **extensão (4080) orchestra**; o **servidor só faz retrieval**. Se o servidor cair, a extensão ainda ensina (sem a fonte do SO). Degradação graciosa, não erro.

4. Componentes (5 unidades, cada uma com um papel)

- 1. **Watcher** (extensão) — escuta o editor, decide *quando* vale comentar (debounce + fim de bloco), evita repetir. Não sabe de LLM nem de SO. *Base: CodingGenie*.
- 2. **Coletor de contexto** (extensão) — extrai o trecho relevante (função/seleção + linguagem). Interface: editor → `{ code, lang, file }`.
- 3. **Serviço de retrieval** (servidor) — recebe o trecho, devolve Q&A do StackOverflow com citação. Isolado e testável sozinho com `curl`.
- 4. **Compositor + Ollama** (extensão) — junta `código + trechos do SO` num *prompt de professor*, chama o Qwen3 14B, recebe a dica didática.
- 5. **Painel "Professor"** (webview) — só renderiza: a dica, o porquê e o link "ver no StackOverflow". Sem lógica de negócio.

Localização do painel: **barra lateral secundária (direita)** do VSCode, com ícone próprio 🏠; o usuário pode reposicionar (esquerda/baixo).

5. Fluxo de dados e gatilhos

Gatilhos: pausa de digitação (~1.5s), `onSave`, ou fim de um bloco sintático. Mais um comando manual "comentar agora".

Fluxo: gatilho → Coletor monta `{code, lang, file}` → (em paralelo) [Serviço de retrieval: top Q&A do SO] + [Compositor monta o prompt] → Ollama (Qwen3 14B, system prompt de professor + contexto do SO) → parse → Painel renderiza (observação + porquê + citação).

Anti-irritação (requisito de 1ª classe): - *Cooldown*: no máximo 1 dica por intervalo configurável. - *Dedup*: não repete dica para trecho inalterado. - Botão **mudo / pausar professor**. - *Limiar de relevância*: o modelo pode responder "nada a comentar" → painel fica quieto. - Toda chamada é **assíncrona**; nunca bloqueia a digitação.

Estilo pedagógico: explica o *porquê* + idioma correto + um empurrão para você mesmo ajustar (ensina, não só corrige). Modo socrático fica como opção futura.

6. Tratamento de erro / degradação graciosa

- **Servidor de retrieval fora** → pula a busca, ensina só com o modelo; painel sinaliza "sem fonte SO agora".
- **Ollama ocupado/lento** → enfileira ou descarta; **nunca trava o editor**; mostra "pensando..." e depois o resultado, ou silencia.
- **Resposta inválida do modelo** (sem JSON/sem sentido) → não mostra nada. Falha em silêncio; jamais spam.
- *Cooldown* protege a 4080 de sobrecarga.

7. Testes

- **Serviço de retrieval:** testes (pytest/ `curl`) com consultas conhecidas no SO `.zim` (query → hit esperado).
- **Lógica do Watcher:** debounce/dedup/cooldown como funções puras, testadas isoladamente.
- **Composição de prompt:** snapshot tests.
- **E2E manual:** digitar um caso "ensinável" conhecido (ex.: `open()` sem `with`) e esperar a dica no painel.

8. Reaproveitamento de open-source (reduz trabalho e falhas)

Peça	Projeto	Licença	Uso
Motor proativo (gatilho/contexto)	CodingGenie	MIT	Fonte de código a adaptar
Prompts de pedagogia	GPTutor	MIT	Referência de explicação didática
Scaffold do painel	vscode-extension-samples / webview-view	MIT	Base da webview
Busca no SO <code>.zim</code>	python-libzim · llm-tools-kiwix	a confirmar	Retrieval com citações
LLM	Ollama + Qwen3 14B	—	Já instalado

Nota estratégica: o **Continue está congelado/read-only** (adquirido pela Cursor, jun/2026; ainda funciona). Por isso a extensão será **self-contained** e o CodingGenie é usado como **fonte de código MIT**, não como dependência de runtime. Bate com a escolha do painel lateral (em vez do chat do Continue). **Confirmar a licença do `llm-tools-kiwix`** na fase de plano.

9. Faseamento (fatias verticais)

1. **Fatia 1 (prova de conceito):** gatilho debounce → Qwen3 14B → painel lateral. **Sem SO.** Valida a "sensação" do proativo e o estilo pedagógico.
2. **Fatia 2:** serviço de retrieval do SO no servidor + citação no painel.
3. **Fatia 3:** anti-irritação completa (cooldown, dedup, mudo, limiar).
4. **Fatia 4 (futuro):** RAG vetorial do SO (prioridade nº4 do acervo-llm — "calcula 1x, usa sempre").

10. Hardware / topologia

- Extensão + Ollama (Qwen3 14B): **máquina 4080** (onde se programa).
- Serviço de retrieval: **servidor da Francesca** (perto do `.zim`, via Tailscale/LAN).

11. Questões em aberto

- Estilo pedagógico padrão: explicativo vs socrático (decidir na Fatia 1).
- Linguagem-alvo inicial: **Python** (projetos atuais) — confirmar.
- Nome e ícone definitivos da extensão.
- Licença do `llm-tools-kiwix` (se não for permissiva, usar `python-libzim` direto).